To proxy requests for all unknown routes in Next.js, you can use a **`proxy.ts` (or `proxy.js`) file** at your project's root level. The `proxy` function in this file will intercept requests and can rewrite them to an external URL. Next.js +4

This method runs before any Next.js page or API route handling, allowing you to catch paths that don't match existing application routes. YouTube

Implementation Steps

1. **Create the Proxy file:** In your project's root directory (or in the `src` directory, at the same level as `app` or `pages`), create a file named `proxy.ts` (or `proxy.js`).
2. **Add Proxy Logic:** Inside the `proxy` file, add logic to check if the incoming request path should be handled by an internal route. If not, use `NextResponse.rewrite()` to point the request to your external service URL.

typescript

```typescript
// proxy.ts
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

export async function proxy(request: NextRequest) {
  const { pathname } = request.nextUrl;

  // Define a list of known route prefixes or use a more comprehensive check
  const knownRoutes = ['/', '/about', '/contact', '/api']; // Add all your app's base r

  // If the request path does not start with any known route prefix, rewrite it
  // Note: This is a simplified check. A more robust solution might require a check
  // against all existing file system routes.
  if (!knownRoutes.some(route => pathname.startsWith(route))) {
    const externalUrl = new URL(pathname, 'https://www.yourexternaldomain.com');
    return NextResponse.rewrite(externalUrl);
  }

  // If it matches a known route, let Next.js handle it normally
  return NextResponse.next();
}
```

3. **Configure the Matcher (Optional but Recommended):** Use the `config.matcher` export to run the proxy function for all paths, which is necessary to catch "unknown" routes.

typescript

```typescript
// proxy.ts (continued)
export const config = {
  matcher: [
    '/:path*', // Match all paths
    '/'
  ],
};
```

Next.js +4

Alternative using `next.config.js` rewrites

For a simpler, more declarative approach that doesn't require complex logic in a `proxy` file, you can use the `rewrites` function in `next.config.js` to create a fallback. LogRocket Blog

javascript

```javascript
// next.config.js
/** @type {import('next').NextConfig} */
const nextConfig = {
  async rewrites() {
    return [
      {
        source: '/:path*', // Match any path
        destination: 'https://www.yourexternaldomain.com*', // Proxy to external URL
        // Make sure this rewrite runs *after* all Next.js page/API routes have been chec
        // The default behavior is to check filesystem routes first.
      },
    ];
  },
};

module.exports = nextConfig;
```

Important Consideration: The `proxy` (formerly middleware) approach offers more fine-grained control and access to the request object, but the `rewrites` feature is generally simpler for basic URL mapping. Next.js automatically attempts to match filesystem routes before applying a general rewrite like `/:path*`. This ensures that your Next.js pages and API routes are served correctly, and only truly "unknown" routes fall through to the external proxy destination.